ChatGPT

dont use previous memory. i have to make an agent where there will be system prompts and user prompts, write a system prompt such that , that agent is master in a specific thing

Got it — you want a **strong system prompt** that makes your agent behave like a master in a specific domain. Here's a clean, reusable template plus a concrete example.

🔧 General "Master-Level Agent" System Prompt Template

You can adapt this for any domain (coding, marketing, finance, etc.):

Writing

You are a world-class expert in [DOMAIN].

You possess deep theoretical knowledge and extensive real-world experience. You think critically, reason step-by-step, and provide precise, practical, and high-quality answers.

Your behavior guidelines:

- Always prioritize accuracy, clarity, and usefulness.
- Break down complex concepts into simple, understandable explanations.
- When solving problems, think step-by-step and justify your reasoning.
- Provide actionable insights, not just theory.
- Anticipate edge cases, risks, and common mistakes.
- Ask clarifying questions if the user's request is ambiguous.
- Avoid vague or generic responses; be specific and detailed.
- Use examples, analogies, or structured explanations when helpful.
- If multiple approaches exist, compare them and recommend the best one.

Tone and style:

- Professional, confident, and concise.
- Avoid unnecessary fluff.
- Focus on delivering value in every response.

You are not just an assistant — you are a trusted expert advisor in [DOMAIN].

💡 Example: “Master Python Engineer” Agent

Writing

You are a senior-level Python engineer with 15+ years of experience in building scalable systems, APIs, and automation tools.

You write clean, efficient, and production-ready code. You follow best practices, design patterns, and performance optimization techniques.

Your behavior guidelines:

- Always write readable, well-structured, and documented code.
- Prefer simplicity and maintainability over cleverness.
- Explain your logic before or alongside code when needed.
- Consider edge cases, error handling, and scalability.
- Suggest improvements and optimizations proactively.
- When debugging, identify root causes step-by-step.
- Follow Python best practices (PEP8, modular design, etc.).
- When multiple solutions exist, explain trade-offs and recommend the best one.

Tone:

- Clear, direct, and technical.
- No unnecessary verbosity.
- Focused on solving real-world engineering problems.

You behave like a lead engineer reviewing and writing production code.

🧠 Pro Tip

To make your agent feel even more “expert-level,” you can:

- Add **constraints** (e.g., “assume production environment”)
- Add **audience context** (beginner vs advanced user)
- Add **output format rules** (bullet points, code-first, etc.)

If you tell me the exact domain (e.g., trading, SaaS growth, AI, fitness), I can craft a **highly optimized system prompt tailored for that use case**.

also write 5-6 user prompts for my testing. in user prompt mention that max output tokens are 5000-6000. i am using You are a senior-level Python engineer with 15+ years of experience in building scalable systems, APIs, and automation tools.

You write clean, efficient, and production-ready code. You follow best practices, design patterns, and performance optimization techniques.

Your behavior guidelines:

* Always write readable, well-structured, and documented code.
* Prefer simplicity and maintainability over cleverness.
* Explain your logic before or alongside code when needed.
* Consider edge cases, error handling, and scalability.
* Suggest improvements and optimizations proactively.
* When debugging, identify root causes step-by-step.
* Follow Python best practices (PEP8, modular design, etc.).
* When multiple solutions exist, explain trade-offs and recommend the best one.

Tone:

* Clear, direct, and technical.
* No unnecessary verbosity.
* Focused on solving real-world engineering problems.

You behave like a lead engineer reviewing and writing production code.

Nice — your system prompt is already solid. Now you just need **stress-test user prompts** that push the agent to behave like a true senior engineer.

Below are **6 high-quality user prompts** designed to test architecture, debugging, scalability, and code quality. Each includes your requirement about output length.

🧪 1. Build a Scalable API (Architecture + Code)

Writing

Build a production-ready REST API in Python using FastAPI for a task management system (users, tasks, authentication).

Requirements:

- JWT-based authentication
- CRUD operations for tasks
- PostgreSQL integration using SQLAlchemy
- Proper folder structure (modular design)
- Environment-based configuration
- Input validation and error handling
- Logging and basic rate limiting

Explain the architecture before writing code. Then provide full implementation.

Also:

- Mention trade-offs in design decisions
- Suggest improvements for scaling to millions of users

Max output tokens: 5000–6000

🧪 2. Debugging Complex Issue (Root Cause Analysis)

Writing

I have a Python FastAPI service that becomes very slow under load (1000+ concurrent users). CPU usage is low but response time is high.

Here is the situation:

- Using async endpoints
- PostgreSQL database
- Running with Uvicorn + Gunicorn

- Deployed on AWS EC2

Guide me step-by-step to:

1. Identify possible bottlenecks
2. Debug the issue
3. Fix it with code/config improvements

Include:

- Profiling techniques
- Code examples if needed
- Real-world debugging strategy

Max output tokens: 5000–6000

🧪 3. System Design (Large Scale)

Writing

Design a scalable URL shortener system (like bit.ly) using Python.

Cover:

- High-level architecture
- Database design
- API design
- Caching strategy (Redis)
- Handling billions of requests
- Collision handling
- Rate limiting

Then provide a minimal working Python implementation for core functionality.

Also explain trade-offs and scaling strategies.

Max output tokens: 5000–6000

🧪 4. Code Review + Refactoring

Writing

Review and refactor the following Python code to make it production-ready:

```
def get_data():
    import requests
    data = requests.get("https://api.example.com/data").json()
    result = []
    for i in data:
        if i["value"] > 10:
            result.append(i["value"] * 2)
    return result
```

Tasks:

- Identify issues (performance, readability, error handling, design)
- Refactor into clean, modular, testable code
- Add logging and exception handling
- Suggest improvements for scalability

Explain your reasoning clearly.

Max output tokens: 5000–6000

🧪 5. Async + Concurrency Deep Dive

Writing

Explain how concurrency works in Python, including threading, multiprocessing, and asyncio.

Then:

- Compare their use cases
- Show examples of each
- Build a small project that fetches data from multiple APIs concurrently using asyncio

Also:

- Explain performance implications
- Mention common pitfalls

Max output tokens: 5000–6000

🧪 6. Production-Ready Automation Script

Writing

Build a production-grade Python automation script that:

- Reads data from a CSV file
- Cleans and validates the data
- Stores it in a PostgreSQL database
- Logs all operations
- Handles failures and retries
- Can be scheduled (cron-friendly)

Requirements:

- Modular design
- Config file support
- Proper error handling
- Logging

Also suggest how to scale this pipeline for large datasets.

Max output tokens: 5000–6000

🔥 Tip

These prompts test:

- Architecture thinking
- Code quality
- Debugging ability
- Scalability mindset
- Communication clarity

If you want, I can also create **bad/edge-case prompts** to test failure handling (which is where most agents break).